COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/15/2026	**DATE SUBMITTED:**	10/15/2026
NAME:	Jose Martin C. Rodulfa	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13 **Parallelism and Multithreading**

1. PROCEDURE OUTPUT

Code:

```
#include <iostream>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
return 0;
}
```

Rodulfa:

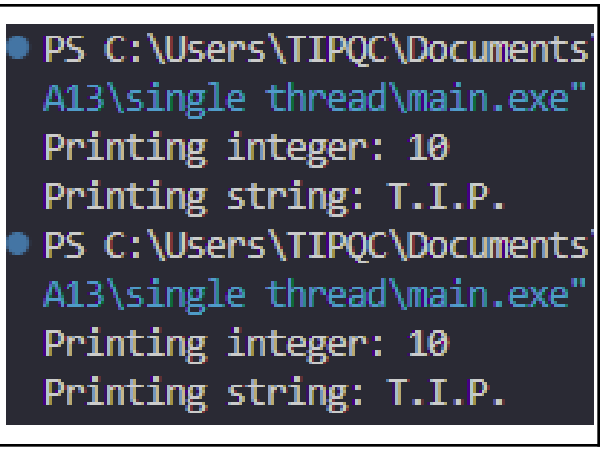
```
Printing integer: 10
Printing string: T.I.P.
 *  Terminal will be reuse

 *  Executing task: C:/Wir
onversion -Wsign-conversio
 -Wextra -Wpedantic -Wshad
d\Debug\singleThread.o -o

 *  Terminal will be reuse

 *  Executing task: C:/Wir

Printing integer: 10
Printing string: T.I.P.
```

Panti:	

ONE-THREAD

ANALYSIS:

Utilizing one thread in a program works by creating a callable function/method (depending on 1 of the 3 chosen techniques) and then invoking said function/method using std::thread and including the function name and variable to be added as parameters to the invocation. We also included a .join function in our program, which is used to essentially tell our main driver function to “wait” until the specific thread finishes initializing. Although the effects of this function are not really felt in this example, its significance is not to be ignored and is incredibly useful when dealing with several threads in one program.

Code:

```cpp
#include <iostream>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

Rodulfa:

```
2 :  Computer
0 :  T.I.P.
1 :  Competent
3 :  Engineers
 *  Terminal will be
```

```
1 :  Competent
2 :  Computer
0 :  T.I.P.
3 :  Engineers
```

Panti:

```
1 : Competent2 : Computer3 : Engineers


0 : T.I.P.
```

```
1 : Competent2 : Computer0 : T.I.P.

3 : Engineers
```

MULTI-THREAD

ANALYSIS:

This program is an example of multithreading in C++, where we used the function pointer technique of creating a callable function, and then passing multiple parameters to the function (via creating a vector of several string variables), essentially creating multiple threads. One for loop is added to label which string variable was added to the thread first (visualizing difference in prioritization/allocation of memory for different variables. A second for loop is included which uses the aforementioned .join function. This function instructs the driver function to "wait" for one thread to finish its execution BEFORE initializing and processing another thread. This is what creates the difference in first and last printed outputs for both of us, as our computers prioritize different variables, both locally (our own devices) and with each other (output of Rodulfa vs. output of Panti).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PART A:

1. **Write a definition of multithreading and its advantages/disadvantages.**

- **Rodulfa:** Multithreading is the technique of using a callable function to handle several different variables via the std::thread library. It essentially applies concurrency on the different variables by having a shared function, but separating the variables used into so-called "threads". This is good for making sure that accidental variable editing/tampering will not happen between the different parameters, at the cost of "data racing", where variables are allocated and prioritized differently every time the program is run.

- **Panti:** Multithreading allows users to run multiple tasks at the same time without causing an overload by alternating threads between cores for better optimization. The CPU can handle more tasks at once but can also be hindering due to the racing of tasks to be accomplished first, causing complex errors to occur.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

- **Rodulfa:** Multithreading can be used when you (1) Want to use only 1 function but with multiple different variables, (2) If you want to utilize the most of your devices CPU cores in running the program, and (3) Taking the load off of key functions/programs by interconnecting different methods together to handle data processing and computation together. Some real-world examples of these features in action are the GUI's of applications, Database Processing and the computation of datasheets, and File Downloaders/Managers.

- **Panti:** One use is by implementing into game engines and graphics processing, allowing for smoother gameplay and rendering. Another is in the use of GUIs which are used to render widgets and interactions. Lastly it can be into web servers or APIs, where it can control the user requests without clogging the server.

3. **Differentiate between parallelism and concurrency.**

- **Rodulfa:** Parallelism is the method in which processes are handled separately from each other, ensuring variable integrity and avoiding "data race" errors (data sharing one function, causing overloading and variation of data prioritization). On the other hand, concurrency is the technique in utilizing a singular method with several different variables, lessening memory load on the program and allows for maximum utilization of the device's CPU cores in handling processes simultaneously and with multiple data/variables.

- **Panti:** Parallelism is when multiple copies of the same task are run through different data points. While concurrency also has multiple copies of a program but they overlap in accessing through a single data point.

PART B:

```
5   int adder = 10;
6
7   void add(int n){
8       int sum;
9       std::cout<<"Initial input: "<< n << std::endl;
10
11      sum = n + adder;
12      std::cout<<"Final output (added number was 10): "<< sum << std::endl;
13  }
```

```
39  //SUPPLEMENTARY
40      std::thread t1(add, 6);
41      t1.join();
42
43      std::thread t2(add, 7);
44      t2.join();
45
46      std::thread t3(add, 8);
47      t3.join();
```

```
1 :  Competent 2 :  Computer 3 :  Engineers

0 :  T.I.P.

Initial input: 6
Final output (added number was 10): 16
Initial input: 7
Final output (added number was 10): 17
Initial input: 8
Final output (added number was 10): 18
```

ANALYSIS:

- This was the solution program that I made for Part B, which consists of a global variable "adder" which will be used for the add operation of our new function. The actual "add" callable function which will use the global variable to add with any chosen input variable, and the .join functions in succession with the individual std::thread function calls. What happens is that the succession of result printing for the add function never changes, thanks to the .join initialization AFTER EVERY CALL of the add function. (This was included in the multithreading file, hence the output at the top).

```
int adder = 0;

void add(int value, const std::string &str) {
    adder += value;
    std::string msg = "Thread processed (" + str + "): added " + std::to_string(value);
    std::cout << msg << std::endl;
}
```

```
std::thread t1(add, values[0], s[0]);
std::cout << "Display before T1.join(): global_var = " << gvi << std::endl;
t1.join();
std::cout << "Display after T1.join(): global_var = " << gvi << std::endl << std::endl;


std::thread t2(add, values[1], s[1]);
std::cout << "Display before T2.join(): global_var = " << gvi << std::endl;
t2.join();
std::cout << "Display after T2.join(): global_var = " << gvi << std::endl << std::endl;


std::thread t3(add, values[2], s[2]);
std::cout << "Display before T3.join(): global_var = " << gvi << std::endl;
t3.join();
std::cout << "Display after T3.join(): global_var = " << gvi << std::endl << std::endl;


std::cout << "Final Display: global_var = " << gvi << std::endl;
```

```
Display before T1.join(): global_var = 0
Thread processed (Competent): added 10
Display after T1.join(): global_var = 10

Display before T2.join(): global_var = 10
Thread processed (Computer): added 20
Display after T2.join(): global_var = 30

Display before T3.join(): global_var = 30
Thread processed (Engineers): added 30
Display after T3.join(): global_var = 60

Final Display: global_var = 60
```

ANALYSIS:

- This example shows how multithreading can be used to do two things at once: display some text and increment a global variable. The strings are handled by a number of threads that run asynchronously in the background. As these string threads are running independently of each other, their outputs intersect. The additional operations are all synchronised by calling .join() immediately after starting each thread. This causes the main program to wait for each addition to complete before it continues to the next one.

This results in the global variable being incremented and the output displayed in sequence.

Panti

PART C:

supplePartC.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <thread>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp;
    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
        }
    }

    while (i <= mid) {
        temp.push_back(arr[i++]);
    }

    while (j <= right) {
        temp.push_back(arr[j++]);
    }

    for (int k = 0; k < temp.size(); k++) {
        arr[left + k] = temp[k];
    }
}
```

```cpp
void parallelMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    std::thread leftThread(parallelMergeSort, std::ref(arr), left, mid);
    std::thread rightThread(parallelMergeSort, std::ref(arr), mid + 1, right);

    leftThread.join();
    rightThread.join();

    merge(arr, left, mid, right);
}

int main() {
    std::vector<int> arr = {117, 4859, 7, 1204, 40000};

    std::cout << "Original Array: ";
    for (int num : arr) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    parallelMergeSort(arr, 0, arr.size() - 1);
```

```cpp
    std::cout << "Sorted Array: ";
    for (int num : arr) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

```
Original Array: 117 4859 7 1204 40000
Sorted Array: 7 117 1204 4859 40000
```

ANALYSIS

- The program implements a multi-threaded Merge Sort algorithm by applying a divide-and-conquer strategy. Each thread recursively sorts its section concurrently across available CPU cores. Calling join() ensures that both sub-threads finish sorting before the main thread merges the ordered halves back into a single sorted array.

Rodulfa

```cpp
void parallel_merge_sort(std::vector<int>& arr, int left, int right, int depth = 0) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    if (depth < 2) {
        std::thread left_thread(parallel_merge_sort, std::ref(arr), left, mid, depth + 1);
        std::thread right_thread(parallel_merge_sort, std::ref(arr), mid + 1, right, depth + 1);

        left_thread.join();
        right_thread.join();
    } else {
        parallel_merge_sort(arr, left, mid, depth + 1);
        parallel_merge_sort(arr, mid + 1, right, depth + 1);
    }

    merge(arr, left, mid, right);
}
```

```cpp
int main() {
    std::vector<int> arr = {38, 27, 43, 3, 9, 82, 10, 19, 50, 12};

    std::cout << "Original array:\n";
    for (int num : arr) {
        std::cout << num << " ";
    }
    std::cout << "\n\n";

    parallel_merge_sort(arr, 0, arr.size() - 1);

    std::cout << "Sorted array:\n";
    for (int num : arr) {
        std::cout << num << " ";
    }
    std::cout << "\n";

    return 0;
}
```

```
Original array:
38 27 43 3 9 82 10 19 50 12

Sorted array:
3 9 10 12 19 27 38 43 50 82
```

ANALYSIS:

- This is a multithread implemented into Merge Sort. This version wraps the recursive calls in thread tasks so that the left and right halves of the array can be sorted in parallel on separate CPU cores. This leads to a reduction in total execution time for large datasets by leveraging the power of parallel hardware. The join() call on both of the sub-threads acts as a synchronisation point where the main thread will

block until both of the sub-threads have completed their sort. Only then will the merge function be called with two fully sorted halves. As spawning new OS threads comes with a certain amount of system overhead (context switching, memory allocation etc), multithreading is only applied at the higher level partition points where the data set is large. For the smaller sub-arrays the algorithm reverts to the standard recursive sort.

Panti

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: N/A

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: N/A

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: N/A

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.